

BAREMETAL ARCH LINUX SERVER CONFIGURATION



Prepared by
Avishek Paudel
avishekspaudel12@gmail.com



Table of Contents

Introduction.....	3
Virtual Machine Creation and Disk Setup.....	3
Arch Linux Installation	4
Securing the Root Account	5
Admin Users	6
Creating the Other User Accounts.....	6
SSH Server	7
Docker Setup	10
Conclusion	11
References.....	12
Appendix	13

Introduction

For this project I was tasked with setting up an Arch Linux server for a small business called EcoGoods who need an on-premises system to host their website, product database, and related services. The entire setup had to be done through the command line with no any graphical interface at any of point. This report outlines the configuration of the virtual machine, Arch Linux Installation, User Management, Security Configuration and Docker Deployment.

Virtual Machine Creation and Disk Setup

I created the virtual machine in VMware Workstation Pro by giving It 10.6GB of Ram and 6 processors. The storage was set to a 40GB SCSI virtual disk configured as a multiple file as required.

Before partitioning I have verified the internet connectivity by running ip a to confirm the network interface was up with addressing 192.168.182.142 then I also pining 8.8.8.8 to confirm external access was working **(See Appendix A)**.

For the disk, I used cfdisk /dev/sda to create a GPT partition table with three partitions **(See Appendix B)**:

Partition	Size	Type	Purpose
/dev/sda1	512M	EFI System	/boot
/dev/sda2	14G	Linux home	/home
/dev/sda3	25.5G	Linux filesystem	/

After confirming the layout with lsblk **(See Appendix C)**, the partition was formatted:

```
mkfs.fat -F32 /dev/sda1
```

```
mkfs.ext4 /dev/sda2
```

```
mkfs.ext4 /dev/sda3
```

The partitions were then mounted with root first at /mnt by following the root and home subdirectories and also verified again with lsblk. **(See Appendix D).**

Arch Linux Installation

Before installing base system, I refreshed the pacman keyring to resolve PGP signature errors that occurred during initial attempts. This was done by running `pacman-key --init`, `pacman-key --populate archlinux`, and `pacman -Sy archlinux-keyring` **(See Appendix E).**

The base system was then installed successfully using `pacstrap` **(See Appendix F):**

```
pacstrap /mnt base base-devel linux linux-firmware
```

`vim` was installed for configuration file editing. The filesystem table was generated and verified using `genfstab` and `cat /mnt/etc/fstab` which correctly showed all three partitions mapped by UUID **(See Appendix G)**. The system was then entered using `arch-chroot /mnt` **(See Appendix H)**.

Inside the chroot the following configuration was applied **(See Appendix I)**. "Arch Linux follows a minimal installation philosophy where only the essential base packages are installed and the user manually configures everything else from scratch, giving full control over the system but requiring a more hands-on setup process compared to other distributions (Cannon, 2020)

- Time-zone: `ln -sf /usr/share/zoneinfo/Asia/Kathmandu /etc/localtime` followed by `hwclock --systohc`
- Locale: `en_GB.UTF-8 UTF-8` was uncommented in `/etc/locale.gen` using `vim`, then `locale-gen` was run confirming "Generation complete"
- Locale config: `echo "LANG=en_GB.UTF-8" > /etc/locale.conf`
- Hostname: `echo "AvishekUAT" > /etc/hostname`
- Hosts file: `/etc/hosts` was edited to include `127.0.1.1 avishekuat.localdomain avishekuat`

NetworkManager was installed with `pacman -S networkmanager` and I enabled with `systemctl enable NetworkManager` (**See Appendix J**). Then the root password was set by using `passwd` (**See Appendix K**).

For the bootloader GRUB was installed with `pacman -S grub` and the configuration was generated with `grub-mkconfig -o /boot/grub/grub.cfg` which successfully found the Linux image at `/boot/vmlinuz-linux` (See Appendix L). After unmounting with `umount -R /mnt` and rebooting with the ISO disconnected the system booted successfully and presented the login prompt for AvishekUAT (**See Appendix M**). Now the installation was complete.

Securing the Root Account

The root account is a common attack target due to its unrestricted privileges. "Allowing root to log in directly over SSH is considered a significant security risk, since attackers only need to target one account to gain full control. For this reason root SSH access was disabled on this system (Daniel J. Barrett, 2005)." It allows root login directly over SSH which means an attacker only needs to guess or brute-force one password to gain the complete control of the server. Due to this reason the following measures were taken to restrict root access.

At first, a strong password was set on the root account using `passwd` command (**See Appendix N**).

Root SSH login was then disabled by editing `/etc/ssh/sshd_config` and setting:

```
PermitRootLogin no
```

Additionally, su was restricted to member of the wheel group only by editing `/etc/pam.d/su` and uncommenting the line:

```
auth required pam_wheel.so use_uid
```

It helps to prevent from unauthorised privilege escalation through su. Administrative task would restrict to authorized sudo users for improving the accountability.

Admin Users

I created two admin accounts which is one for myself (sysadmin) and one for Ashley the IT manager. Both went into the wheel group so they'd have sudo access (**See Appendix O**):

```
useradd -m -G wheel -s /bin/bash sysadmin
```

```
passwd sysadmin
```

```
useradd -m -G wheel -s /bin/bash ashley
```

```
passwd ashley
```

I used visudo to check the sudoers file and the %wheel All=(All:All) line was already uncommented so sudo was working for both accounts. The reason I used visudo rather than editing the file directly is that it checks syntax before saving that means if you make a typo in the sudoers file and save it normally you can lock yourself out of sudo entirely which is obviously a problem.

Creating the Other User Accounts

Before creating the user, I setup the groups they had need (**See Appendix P**):

```
groupadd developers
```

```
groupadd webdev
```

```
groupadd testers
```

A docker group was created for authorised Docker management. Avery required full access to /srv/ and Docker management permissions. I created him with the developers and docker groups **(See Appendix Q)**:

```
useradd -m -G developers,docker -s /bin/bash avery
```

```
passwd avery
```

Evelynn is the website developer so she only needs to drop files into /srv/http and the requirements says that she will not use the CLI at all. So, I created her account with the webdev group SSH configuration restricted to Evelynn to SFTP-only access within a chroot environment **(See Appendix R)**:

```
useradd -m -G webdev -s /bin/bash evelynn
```

```
passwd evelynn
```

James is the software tester. He needs to be able to read through code and run scripts to check they work by he will not be able to edit anything. That is handled through ACL permission on the directory rather than anything at account level **(See Appendix S)**:

```
useradd -m -G testers -s /bin/bash james
```

```
passwd james
```

I also added administrative accounts to the developer group **(See Appendix T)**:

```
usermod -aG developers sysadmin
```

```
usermod -aG developers ashley
```

SSH Server

I installed OpenSSH enabled it to start on boot and I started it directly **(See Appendix U)**:

```
pacman -S openssh
```

```
systemctl enable sshd
```

```
systemctl start sshd
```

Then I went into `/etc/ssh/sshd_config` and made a few changes which are shown below (**See Appendix V**):

```
PermitRootLogin no
```

```
PubkeyAuthentication yes
```

```
PasswordAuthentication yes
```

```
X11Forwarding no
```

```
MaxAuthTries 4
```

```
AllowUsers sysadmin ashley avery james evelynn
```

This created an SSH whitelist for authorised users. So, even if someone created a new account or compromised a server account they could not in into the SSH. `MaxAuthTries 4` helps to reduce the brute-force login attempts. X11 forwarding was disabled because the server will not operate without a graphical interface.

For Evelynnn, I added a match block at the bottom of the config:

```
Match User evelynn
```

```
ForceCommand internal-sftp
```

```
ChrootDirectory /srv/http
```

```
PermitTunnel no
```

```
AllowTcpForwarding no
```

```
X11Forwarding no
```

`ChrootDirectory /srv/http` traps her session inside that folder. She can not even see or nagivate to anything else on the system. That means she is restricted to SFTP-only access with `/srv/http`.

Directory Permission for `/srv/`

I created both directories that were needed (**See Appendix W**):

```
mkdir -p /srv/dev
```

```
mkdir -p /srv/http
```

for /srv/dev I set the group of developer and I used permission 2770 (**See Appendix X**):

```
chown root:developers /srv/dev
```

```
chmod 2770 /srv/dev
```

The 2770 will break down as developers group gets full read/write/execute, nobody else gets anything and the 2 at the front sets the setgid bit. "When the setgid bit is set on a directory any new file or folder created inside it will automatically inherit the group ownership of the parent directory rather than belonging to the creating user's own primary group, which is exactly what is needed for a shared team directory like /srv/dev (Ward, 2021)." That setgid bit is actually really useful here because in any file or folder created inside /srv/dev automatically come into the developer group rather than the creating user's primary group.

For James, I needed him to be able to read files and execute scripts but not write anything. The problem is that standard Linux permissions only support one group per directory so, I cannot set developer to rwx and testers to r-x using normal chmod. That is why ACLs come in (**See Appendix Y**). "Unlike standard Linux permissions which only allow one set of group rules per directory, POSIX ACLs make it possible to assign different permissions to multiple users and groups on the same folder independently, giving much more flexibility over access control (Grünbacher, 2003)."

```
pacman -S acl
```

```
setfacl -R -m g:testers:r-x /srv/dev
```

```
setfacl -R -d -m g:testers:r-x /srv/dev
```

ACLs were applied to read and execute the permission for tester on current and future files. So, new file James cannot write to them.

For /srv/http:

```
chown root:webdev /srv/http
```

```
chmod 2775 /srv/http
```

The 2775 will give the webdev group and also give access so Evelyn can upload files while making sure others can read that as well. This is needed so Nginx can actually serve the website content (**See Appendix Z**).

Docker Setup

I installed and enabled Docker and Docker Compose. "Docker works by packaging applications and everything they need to run into isolated containers at the operating system level, which means they can be deployed consistently across different environments without needing a full virtual machine for each one (Merkel, 2014)." Then, I added the right users to the docker group as well (**See Appendix AA**):

```
usermod -aG docker avery
```

```
usermod -aG docker sysadmin
```

```
usermod -aG docker ashley
```

James and Evelyn were intentionally removed out. The Docker socket gives whoever has access to root level control over the host machine.

I put the compose file together at /opt/docker-stack/docker-compose.yml with three services on an internal bridge network (**See Appendix AB**). MySQL has no ports mapped to the host at all so it is completely unreachable from outside Docker. Nginx was also configured as a reverse proxy for PhpMyAdmin on the /phpMyAdmin route. Database access was restricted to internal container networking. PhpMyAdmin is set up the same way no host-ports only internal network and only accessible through the Nginx reverse proxy.

Nginx is the only container where on port 80 is totally open to the outside world (**See Appendix AC**). It mounts /srv/http as read-only volume so website files placed there by Evelyn show up in the container but Nginx itself can not modify anything. The nginx.conf handles two routes which is / server (website files) and /phpMyAdmin/ proxies requests through to the PhpMyAdmin container.

All the containers were verified as running by using docker compose ps **(See Appendix AD)**. The last thing was giving Avery proper database admin access inside the MySQL **(See Appendix AE)**:

```
GRANT ALL PRIVILEGES ON *.* TO 'avery'@'%' WITH GRANT OPTION;
```

```
FLUSH PRIVILEGES;
```

Both quires came back with queries OK which confirmed me that it is working. Avery can now create and manage database and users within the MySQL instance.

Both the website and PhpMyAdmin were verified as accessible from the host machine browser. **(See Appendix AF)**.

Conclusion

The whole system was successfully configured to meet the requirements by using Linux administration practices. User Permissions, SSH access, Docker management and container networking were configured by using the principle of least privilege. "Throughout this configuration, each user and service was given only the access they needed to do their job and nothing more, which follows the principle of least privilege and helps keep the overall attack surface of the system as small as possible (Jerome H. Saltzer, 1975)." Remote root access was also disabled and internal services were protected from external exposure.

References

Cannon, J., 2020. *Arch Linux Handbook: A Simple, Lightweight Linux Handbook*. 3rd ed. s.l.:Independently Published.

Daniel J. Barrett, R. E. S. R. G. B., 2005. *SSH, the Secure Shell: The Definitive Guide*. 2nd ed. Sebastopol, California: O'Reilly Media.

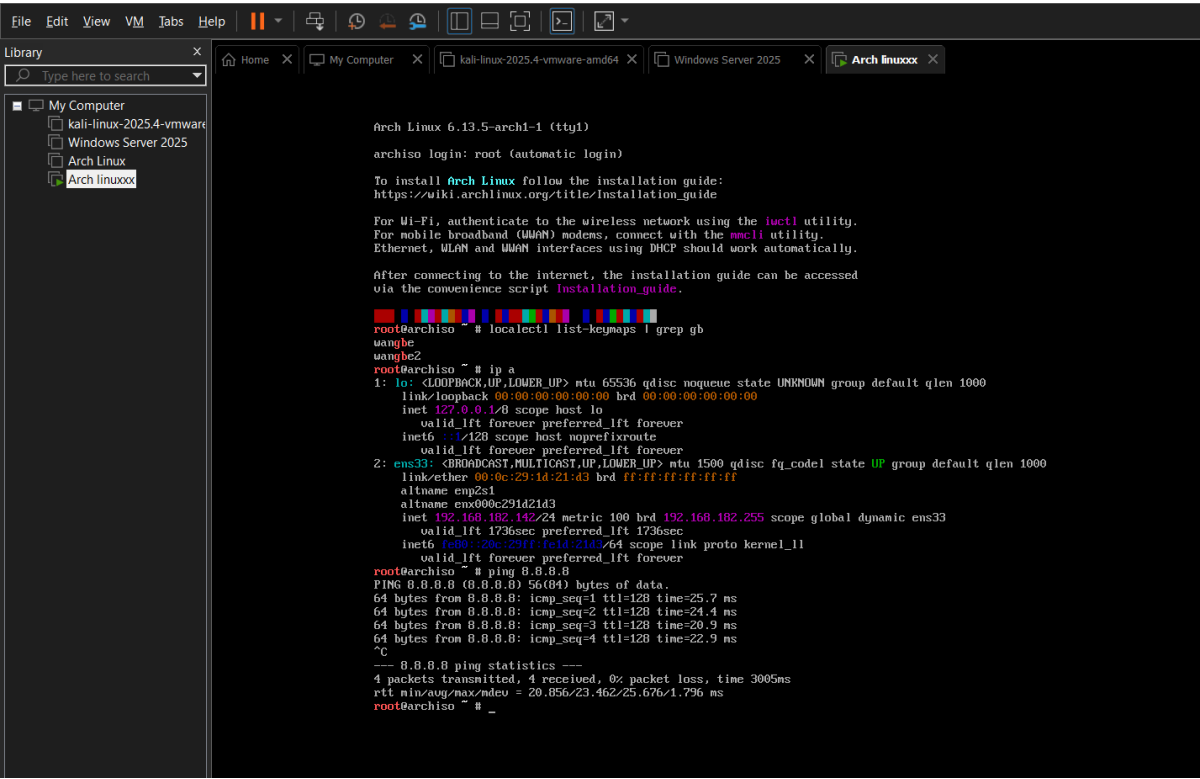
Grünbacher, A., 2003. POSIX access control lists on Linux. *ACM SIGOPS Operating Systems Review*, 37(1), pp. 71-79.

Jerome H. Saltzer, M. D. S., 1975. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), pp. 1278-1308.

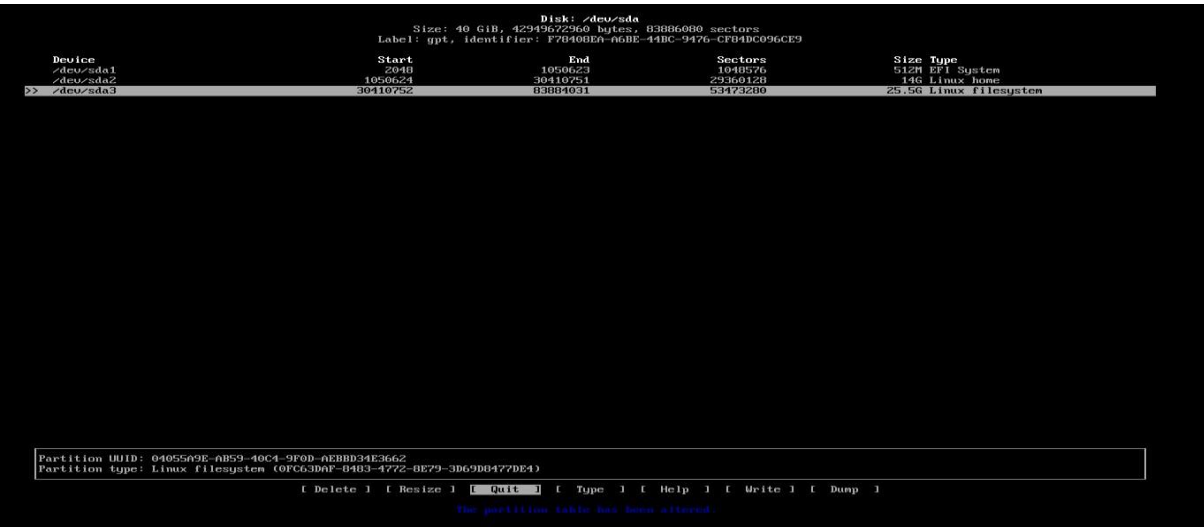
Merkel, D., 2014. Docker: lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239), pp. 2-9.

Ward, B., 2021. *How Linux Works: What Every Superuser Should Know*. 3rd ed. San Francisco: No Starch Press.

Appendix



Appendix A – Network verification: The following screenshot shows the network verification carried out before beginning the installation process.



Appendix B – Partition Layout: The following screenshot shows the completed partition table created using cfdisk on /dev/sda.

```
root@archiso ~ # lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0       7:0      0 836.7M  1 loop /run/archiso/airootfs
sda         8:0      0   40G   0 disk
├─sda1      8:1      0   512M   0 part
├─sda2      8:2      0    14G   0 part
└─sda3      8:3      0   25.5G   0 part
sr0         11:0     1    1.2G   0 rom  /run/archiso/bootmnt
root@archiso ~ # _
```

Appendix C – Lsblk Formatting: The following screenshot shows the output of Lsblk after partitioning but before formatting.

```
root@archiso ~ # mkfs.fat -F32 /dev/sda1
mkfs.fat 4.2 (2021-01-31)
root@archiso ~ # mkfs.ext4 /dev/sda2
mke2fs 1.47.2 (1-Jan-2025)
/dev/sda2 contains a ext4 file system
last mounted on Wed Apr 23 05:42:56 2026
Proceed anyway? (y,n) y
Creating filesystem with 3670016 4k blocks and 917504 inodes
Filesystem UUID: 329496d5-45d1-4832-b832-65f8b287f441
Superblock backups stored on blocks:
    32768, 36864, 163840, 223376, 294912, 813300, 884736, 1665632, 2654208

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

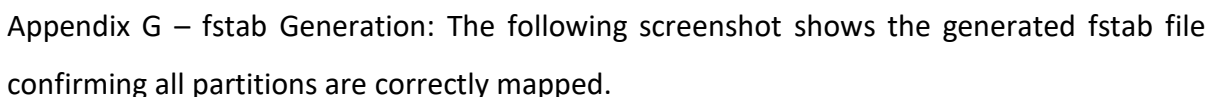
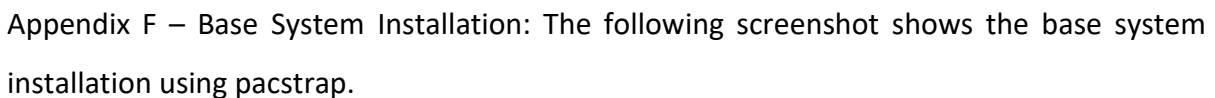
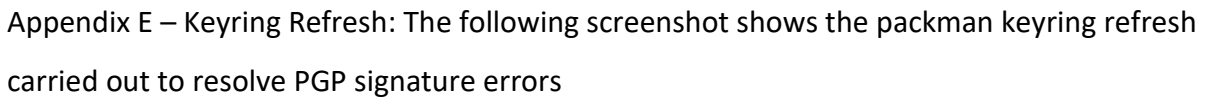
root@archiso ~ # mkfs.ext4 /dev/sda3
mke2fs 1.47.2 (1-Jan-2025)
/dev/sda3 contains a ext4 file system
created on Wed Apr 23 05:40:56 2026
Proceed anyway? (y,n) y
Creating filesystem with 6684160 4k blocks and 1671168 inodes
Filesystem UUID: a24956dd-4315-48a2-9913-90f580a55d02
Superblock backups stored on blocks:
    32768, 36864, 163840, 223376, 294912, 813300, 884736, 1665632, 2654208,
    4996800

Allocating group tables: done
Writing inode tables: done
Creating journal (32768 blocks): done
Writing superblocks and filesystem accounting information: done

root@archiso ~ #
```

```
root@archiso ~ # mount /dev/sda3 /mnt
root@archiso ~ # mkdir -p /mnt/boot /mnt/home
root@archiso ~ # mount /dev/sda1 /mnt/boot
zsh: correct 'mount' to 'moun' [nyae]?
130 root@archiso ~ # mount /dev/sda1 /mnt/boot
root@archiso ~ # mount /dev/sda2 /mnt/home
root@archiso ~ # lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0       7:0      0 836.7M  1 loop /run/archiso/airootfs
sda         8:0      0   40G   0 disk
├─sda1      8:1      0   512M   0 part /mnt/boot
├─sda2      8:2      0    14G   0 part /mnt/home
└─sda3      8:3      0   25.5G   0 part /mnt
sr0         11:0     1    1.2G   0 rom  /run/archiso/bootmnt
root@archiso ~ #
```

Appendix D – Formatting and Mounting: The following Screenshots shows the formatting of each partition and the mounting process.



```
root@archiso ~ # arch-chroot /mnt
```

Appendix H – chroot: The following screenshot shows the arch-chroot command entering the newly installed system.

```
root@archiso /]# ln -sf /usr/share/zoneinfo/Asia/Kathmandu /etc/localtime
root@archiso /]# hwclock --systohc
root@archiso /]# echo "LANG=en_GB.UTF-8: > /etc/locale.conf
> locale-gen
> ^C
root@archiso /]# echo "LANG=en_GB.UTF-8" > /etc/locale.conf
root@archiso /]# locale-gen
Generating locales...
Generation complete.
root@archiso /]#
```

```
[root@archiso /]# echo "AvishekUAT" > /etc/hostname
```

```
Arch Linux
# See hosts(5) for details.
127.0.0.1    localhost
::1         localhost
127.0.0.1    avishekuaat.localdomain avishekuaat

"etc/hosts" SL, 163B written
root@archiso /]#
```

Appendix I – Time-zone, Locale and Hostname: The following screenshots shows the time-zone, locale and hostname configuration which is applied inside the chroot.

```
python: libnft support with the _snack module
tc: uhlptcl support
( 6/19) installing nmap
( 7/19) installing nss
( 8/19) installing libm
( 9/19) installing libnemon
(10/19) installing libcodium
(11/19) installing libpym
(12/19) installing zeromq
Optional dependencies for zeromq
  cpzmq: C++ binding for libzmq
(13/19) installing libteam
(14/19) installing mobile-broadband-provider-info
(15/19) installing duktap
(16/19) installing polkit
(17/19) installing pccclite
Optional dependencies for pccclite
  python: API call trace logging with the pccs-spy
  ccid: USB Chip/Smart Card Interface Devices driver
(18/19) installing upa-supplcmt
(19/19) installing networkmanager
Optional dependencies for networkmanager
  bluez: Bluetooth support
  dhcpcd: alternative DHCP client
  dnsmasq: connection sharing
  firewalld: firewall support
  iptables: connection sharing (installed)
  lsd: upa-supplcmt alternative
  libnme: NBT support
  modemmanager: cellular network support
  nftables: connection sharing (installed)
  openresolv: alternative resolv.conf manager
  pacrunner: PAC proxy support
  polkit: let non-root users control networking (installed)
  ppp: dialup connection support
:: Running post-transaction hooks...
(1/6) Creating system user accounts...
  Creating group 'polkitd' with GID 102.
  Creating group 'pcscd' with GID 969.
  Creating user 'pcscd' (PC/SC Smart Card Daemon) with UID 969 and GID 969.
  Creating user 'polkitd' (User for polkitd) with UID 102 and GID 102.
(2/6) Creating temporary files...
(3/6) Reloading system manager configuration...
  Skipped: Running in chroot.
(4/6) Reloading device manager configuration...
  Skipped: Running in chroot.
(5/6) Arming ConditionNeedsUpdate...
(6/6) Reloading system bus configuration...
  Skipped: Running in chroot.
root@archiso /# _
```

```
root@archiso /# systemctl enable NetworkManager
Created symlink '/etc/systemd/system/multi-user.target.wants/NetworkManager.service' → '/usr/lib/systemd/system/NetworkManager.service'.
Created symlink '/etc/systemd/system/dbus-org.freedesktop.nm-dispatcher.service' → '/usr/lib/systemd/system/NetworkManager-dispatcher.service'.
Created symlink '/etc/systemd/system/network-online.target.wants/NetworkManager-wait-online.service' → '/usr/lib/systemd/system/NetworkManager-wait-online.service'.
root@archiso /# _
```

Appendix J – NetworkManager: The following screenshots show NetworkManager being installed and enabled.

```
root@archiso /# passwd
New password:
Retype new password:
passwd: password updated successfully
root@archiso /#
```

Appendix K – Root Password in Chroot: The following screenshots shows the root password being set inside the chroot environment.

```
passwd: password updated successfully
root@archiso /# pacman -S grub
resolving dependencies...
looking for conflicting packages...

Packages (1) grub-2:2.14-1

Total Download Size: 7.59 MiB
Total Installed Size: 41.49 MiB

:: Proceed with installation? [Y/n] y
:: Retrieving packages...
grub-2:2.14-1-x86_64 7.6 MiB 10.3 MiB/s 00:00 [#####] 100%
(1/1) checking keys in keyring [#####] 100%
(1/1) checking package integrity [#####] 100%
(1/1) loading package files [#####] 100%
(1/1) checking for file conflicts [#####] 100%
(1/1) checking available disk space [#####] 100%
:: Processing package changes...
(1/1) installing grub [#####] 100%
:: Install your bootloader and generate configuration with:
# grub-install ...
# grub-mkconfig -o /boot/grub/grub.cfg
Optional dependencies for grub
  dosfstools: For grub-mkrescue FAT FS and EFI support
  efibootmgr: For grub-install EFI support
  freetype2: For grub-mkfont usage
  fuse3: For grub-mount usage
  libisoburn: Provides convenience for generating grub rescue iso using grub-mkrescue
  libusb: For grub-emu USB support (installed)
  lzop: For grub-mkrescue LZO support
  nttools: For grub-mkrescue FAT FS support
  os-prober: To detect other OSes when generating grub.cfg in BIOS systems
  sd: For grub-emu SBL support
:: Running post-transaction hooks...
(1/1) Arming ConditionNeedsUpdate...
root@archiso /# _
```

```
[root@archiso /]# grub-mkconfig -o /boot/grub/grub.cfg
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-linux
Found initrd image: /boot/initramfs-linux.img
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
[root@archiso /]#
```

Appendix L – GRUB Installation: The following screenshots show GRUB being installed and the boot configuration being generated.

```
Arch Linux 6.19.14-arch1-1 (tty1)

AvishekUAT login:
```

```
AvishekUAT login: root
Password:
[root@AvishekUAT ~]#
```

Appendix M – First Successful Boot: The following screenshot show the system booting successfully into the installed Arch Linux environment for the first time.

```
AvishekUAT login: root
Password:
[root@AvishekUAT ~]#
```

Appendix N – Roots Password on Installed System: The following screenshots shows the root password being changed on the installed system after first boot.

```
[root@AvishekUAT ~]# useradd -m -G wheel -s /bin/bash sysadmin
[root@AvishekUAT ~]# passwd sysadmin
New password:
Retype new password:
passwd: password updated successfully
[root@AvishekUAT ~]# _
```

```
[root@AvishekUAT ~]# useradd -m -G wheel -s /bin/bash ashley
[root@AvishekUAT ~]# passwd ashley
New password:
Retype new password:
passwd: password updated successfully
[root@AvishekUAT ~]#
```

Appendix O – Admin Users created: The following screenshots show the creation of the two administrative user accounts.

```
[root@AvishekUAT ~]# groupadd developers
[root@AvishekUAT ~]# groupadd webdev
[root@AvishekUAT ~]# groupadd testers
[root@AvishekUAT ~]#
```

Appendix P – Groups Created: The following screenshot shows the creation of the required user groups.

```
[root@AvishekUAT ~]# useradd -m -G developers,docker -s /bin/bash avery
[root@AvishekUAT ~]# passwd avery
New password:
Retype new password:
passwd: password updated successfully
[root@AvishekUAT ~]#
```

Appendix Q – Avery Created: The following screenshot shows the creation of Avery's user account.

```
[root@AvishekUAT ~]# useradd -m -G webdev -s /bin/bash evelynn
[root@AvishekUAT ~]# passwd evelynn
New password:
Retype new password:
passwd: password updated successfully
[root@AvishekUAT ~]#
```

Appendix R – Evelynn Created: The following screenshot shows the creation of Evelynn's user account.

```
[root@AvishekUAT ~]# useradd -m -G testers -s /bin/bash james
[root@AvishekUAT ~]# passwd james
New password:
Retype new password:
passwd: password updated successfully
[root@AvishekUAT ~]#
```

Appendix S – James Created: The following screenshot shows the creation of the James’s user account.

```
[root@AvishekUAT ~]# usermod -aG developers sysadmin
[root@AvishekUAT ~]# usermod -aG developers ashley
```

Appendix T – Admins Added to Developer: The following screenshot shows the admin accounts being added to the developers group.

```
looking for conflicting packages...
Packages (1) openssh-10.3p1-1
Total Installed Size: 5.46 MiB
Net Upgrade Size: 0.00 MiB

:: Proceed with installation? [Y/n] y
(1/1) checking keys in keyring
(1/1) checking package integrity
(1/1) loading package files
(1/1) checking for file conflicts
(1/1) checking available disk space
:: Processing package changes...
(1/1) reinstalling openssh
:: Running post-transaction hooks...
(1/6) Creating temporary files...
(2/6) Reloading system manager configuration...
[ 539.72282] systemd-fstab-generator[1395]: Failed to create unit file '/run/systemd/generator/-.mount', as it already exists. Duplicate entry in '/etc/fstab'
?
[ 539.72352] systemd-fstab-generator[1395]: Failed to create unit file '/run/systemd/generator/boot.mount', as it already exists. Duplicate entry in '/etc/fstab'
ab'?
[ 539.72616] systemd-fstab-generator[1395]: Failed to create unit file '/run/systemd/generator/home.mount', as it already exists. Duplicate entry in '/etc/fstab'
ab'?
[ 539.72708] systemd-fstab-generator[1395]: Failed to create unit file '/run/systemd/generator/-.mount', as it already exists. Duplicate entry in '/etc/fstab'
?
[ 539.72794] systemd-fstab-generator[1395]: Failed to create unit file '/run/systemd/generator/boot.mount', as it already exists. Duplicate entry in '/etc/fstab'
ab'?
[ 539.72896] systemd-fstab-generator[1395]: Failed to create unit file '/run/systemd/generator/home.mount', as it already exists. Duplicate entry in '/etc/fstab'
ab'?
(3/6) Reloading user manager configuration...
(4/6) Enqueuing marked services...
(5/6) Arming ConditionNeedsUpdate...
(6/6) Restart a running sshd.service
[root@AvishekUAT ~]# systemctl enable sshd
Created symlink '/etc/systemd/system/multi-user.target.wants/sshd.service' → '/usr/lib/systemd/system/sshd.service'.
[ 549.23762] systemd-fstab-generator[1461]: Failed to create unit file '/run/systemd/generator/-.mount', as it already exists. Duplicate entry in '/etc/fstab'
?
[ 549.23963] systemd-fstab-generator[1461]: Failed to create unit file '/run/systemd/generator/boot.mount', as it already exists. Duplicate entry in '/etc/fstab'
ab'?
[ 549.24064] systemd-fstab-generator[1461]: Failed to create unit file '/run/systemd/generator/home.mount', as it already exists. Duplicate entry in '/etc/fstab'
ab'?
[ 549.24065] systemd-fstab-generator[1461]: Failed to create unit file '/run/systemd/generator/-.mount', as it already exists. Duplicate entry in '/etc/fstab'
?
[ 549.24074] systemd-fstab-generator[1461]: Failed to create unit file '/run/systemd/generator/boot.mount', as it already exists. Duplicate entry in '/etc/fstab'
ab'?
[ 549.24076] systemd-fstab-generator[1461]: Failed to create unit file '/run/systemd/generator/home.mount', as it already exists. Duplicate entry in '/etc/fstab'
ab'?
[root@AvishekUAT ~]# systemctl start sshd
[root@AvishekUAT ~]# _
```

Appendix U – OpenSSH Installed and Started: The following screenshot shows OpenSSH being Installed, enabled and started.

```
Arch Linux x

#AllowTcpForwarding yes
#GatewayPorts no
#X11Forwarding no
#X11DisplayOffset 10
#X11UseLocalhost yes
#PermitTTY yes
#PrintMotd yes
#PrintLastLog yes
#TCPKeepAlive yes
#PermitUserEnvironment no
#Compression delayed
#ClientAliveInterval 0
#ClientAliveCountMax 3
#UseDNS no
#PidFile /run/sshd.pid
#MaxStartups 10:30:100
#PermitTunnel no
#ChrootDirectory none
#VersionAddendum none

# no default banner path
#Banner none

# override default of no subsystems
Subsystem sftp /usr/lib/ssh/sftp-server

# Example of overriding settings on a per-user basis
#Match User anoncvs
#    X11Forwarding no
#    AllowTcpForwarding no
#    PermitTTY no
#    ForceCommand cvs server
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication yes
X11Forwarding no
MaxAuthTries 4
AllowUsers sysadmin ashley avery james evelynn

Match User evelynn
ForceCommand internal-sftp
ChrootDirectory /srv/http
PermitTunnel no
AllowTcpForwarding no
X11Forwarding no
```

Appendix V – SSH Configuration: The following screenshot shows the security setting applied to /etc/ssh/sshd_config.

```
[root@AvishekUAT ~]# mkdir -p /srv/dev
[root@AvishekUAT ~]# mkdir -p /srv/http
[root@AvishekUAT ~]#
```

Appendix W - /srv/Directories Created: The following screenshot shows the creation of the required directories under /srv/.

```
[root@AvishekUAT ~]# chown root:developers /srv/dev
[root@AvishekUAT ~]# chmod 2770 /srv/dev
[root@AvishekUAT ~]# _
```

Appendix X - /srv/dev Permission: The following screenshot shows the ownership and permission applied to /srv/dev.

```
[root@AvishekUAT ~]# pacman -S acl
warning: acl-2.3.2-2 is up to date -- reinstalling
resolving dependencies...
looking for conflicting packages...

Packages (1) acl-2.3.2-2

Total Installed Size: 0.33 MiB
Net Upgrade Size: 0.00 MiB

:: Proceed with installation? [Y/n] y
(1/1) checking keys in keyring
(1/1) checking package integrity
(1/1) loading package files
(1/1) checking for file conflicts
(1/1) checking available disk space
:: Processing package changes...
(1/1) reinstalling acl
:: Running post-transaction hooks...
(1/1) Archiving ConditionNeedsUpdate...
[root@AvishekUAT ~]# setfacl -R -m g:testers:r-x /srv/dev
[root@AvishekUAT ~]# setfacl -R -d -m g:testers:r-x /srv/dev
[root@AvishekUAT ~]#
```

100%

100%

100%

100%

Appendix Y – ACL for Testers: The following screenshot shows the ACL package being installed and the tester permission being applied to /srv/dev.

```
[root@AvishekUAT ~]# chown root:webdev /srv/http
[root@AvishekUAT ~]# chmod 2775 /srv/http
[root@AvishekUAT ~]#
```

Appendix Z - /srv/http Permissions: The following screenshot shows the ownership and permission applied to /srv/http.

```
[root@AvishekUAT ~]# usermod -aG docker avery
[root@AvishekUAT ~]# usermod -aG docker sysadmin
[root@AvishekUAT ~]# usermod -aG docker ashley
[root@AvishekUAT ~]# mkdir -p /opt/docker-stack
[root@AvishekUAT ~]# _
```

```
[root@AvishekUAT ~]# usermod -aG docker avery
[root@AvishekUAT ~]# usermod -aG docker sysadmin
[root@AvishekUAT ~]# usermod -aG docker ashley
[root@AvishekUAT ~]# _
```

Appendix AA – Docker Group Assignment: The following screenshots shows the docker group being assigned to the appropriate user and the Docker stack directory being created.

```
services:
  mysql:
    image: mysql:8.0
    container_name: mysql_db
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: kali
      MYSQL_DATABASE: appdb
      MYSQL_USER: avery
      MYSQL_PASSWORD: kali
    volumes:
      - mysql_data:/var/lib/mysql
    networks:
      - internal
  phpmyadmin:
    image: phpmyadmin:latest
    container_name: phpmyadmin
    restart: unless-stopped
    environment:
      PMA_HOST: mysql
      PMA_PORT: 3306
    networks:
      - internal
  nginx:
    image: nginx:latest
    container_name: nginx_web
    restart: unless-stopped
    ports:
      - "80:80"
    volumes:
      - /srv/http:/usr/share/nginx/html:ro
      - /opt/docker-stack/nginx.conf:/etc/nginx/nginx.conf:ro
    networks:
      - internal
networks:
  internal:
    driver: bridge
volumes:
  mysql_data:
~
~
~
~
~
~
~
"/opt/docker-stack/docker-compose.yml" 39L, 879B
```

Appendix AB – Docker Compose File: The following screenshot shows the docker-compose.yml file configured for the three container services.


```
mysql> GRANT ALL PRIVILEGES ON *.* TO 'avery'@'%' WITH GRANT OPTION;
Query OK, 0 rows affected (0.01 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.01 sec)

mysql> EXIT;
Bye
[root@avishekUAT docker-stack]#
```

Appendix AE – MySQL Admin Grant: The following screenshot shows Avery being granted full database administrator privileges inside the MySQL container.



Appendix AF – PhpMyAdmin Accessible through Reverse Proxy: The following screenshot shows PhpMyAdmin being accessed through the Nginx reverse proxy on the /phpmyadmin/ route from the host machine browse.